

DISPENSA DELLA LEZIONE

Fetch in React

Recuperare dati da API, organizzare le richieste e gestire caricamenti ed errori.



CAPITOLO 1

Fetch semplice

NOTA

`fetch()` è una funzione JavaScript che invia una richiesta a un'API, cioè a un servizio che fornisce dati. La richiesta è asincrona: il risultato arriva dopo un breve tempo e non immediatamente.

Senza usare `await`, si usa `then()`. Il primo `then()` legge la risposta, mentre il secondo riceve i dati già convertiti in un oggetto JavaScript.

In React i dati ricevuti da una fetch vengono di solito salvati nello state. Quando lo state cambia, React aggiorna automaticamente la parte di interfaccia che usa quel valore.

esempio

```
import { useState } from "react";

function App() {
  const [post, setPost] = useState(null);

  function loadPost() {
    fetch("https://jsonplaceholder.typicode.com/posts/1")
      .then((response) => response.json())
      .then((data) => {
        setPost(data);
      });
  }

  return (
    <div>
      <button onClick={loadPost}>Carica post</button>

      {post && (
        <article>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      )}
    </div>
  );
}
```

La fetch parte soltanto al click del bottone. `response.json()` converte la risposta ricevuta in dati JavaScript, poi `setPost(data)` salva il post nello state.

CAPITOLO 2

Async e await

NOTA

`async` e `await` sono un modo alternativo per gestire le operazioni asincrone. Permettono di scrivere una fetch con una sintassi simile a quella delle istruzioni normali, dall'alto verso il basso.

La parola chiave `async` viene scritta prima della funzione. All'interno di una funzione `async`, `await` sospende l'esecuzione della funzione finché la Promise indicata non ha prodotto un risultato.

Con `await fetch(...)` si aspetta la risposta del server. Con `await response.json()` si aspetta la conversione della risposta in dati JavaScript. Solo dopo questi due passaggi i dati possono essere salvati nello state.

esempio

```
import { useState } from "react";

function App() {
  const [post, setPost] = useState(null);

  async function loadPost() {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/posts/1"
    );

    const data = await response.json();

    setPost(data);
  }

  return (
    <div>
      <button onClick={loadPost}>Carica post</button>

      {post && (
        <article>
          <h2>{post.title}</h2>
          <p>{post.body}</p>
        </article>
      )}
    </div>
  );
}
```

Il risultato finale è uguale all'esempio con `then()`, ma la struttura della funzione può risultare più leggibile. `await` non blocca tutta l'applicazione: mette in attesa solo la funzione `loadPost()`, mentre React può continuare a gestire l'interfaccia.

CAPITOLO 3

Inviare un input a Mistral

NOTA

Le API di Mistral permettono di inviare messaggi a un modello AI e ricevere una risposta. Per ottenere una risposta testuale si invia una richiesta **POST** all'endpoint chat completions.

Un campo **input** controllato da React salva ciò che l'utente scrive nello state. Al click del bottone, quel valore viene inserito nel body della richiesta e inviato a Mistral.

Una richiesta **POST** invia dati al server. Il body deve essere convertito in JSON con **JSON.stringify()**. Gli header comunicano al server che il contenuto è JSON e inviano l'API key necessaria per autorizzare la richiesta.

Negli esempi viene usato il valore segnaposto **LA_TUA_API_KEY**. In seguito la chiave verrà letta dal file **.env**, separandola dal codice del componente.

esempio

```
import { useState } from "react";

function App() {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");

  async function askMistral() {
    const response = await fetch(
      "https://api.mistral.ai/v1/chat/completions",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          "Authorization": "Bearer LA_TUA_API_KEY"
        },
        body: JSON.stringify({
          model: "mistral-small-latest",
          messages: [
            {
              role: "user",
              content: question
            }
          ]
        })
      }
    );

    const data = await response.json();

    setAnswer(data.choices[0].message.content);
  }

  return (
    <div>
      <input
        type="text"
        value={question}
        onChange={(event) => setQuestion(event.target.value)}
        placeholder="Scrivi una domanda"
      />

      <button onClick={askMistral}>Invia a Mistral</button>
    </div>
  );
}
```

```
      {answer && <p>{answer}</p>}  
    </div>  
  );  
}
```

Il valore di **question** viene aggiornato a ogni modifica dell'input. Mistral restituisce la risposta nell'array **choices**, quindi il testo prodotto dal modello si legge con **data.choices[0].message.content**.

CAPITOLO 4

Usare il file .env

NOTA

Il file `.env` contiene variabili di ambiente, cioè valori esterni al codice React. Può essere usato per separare configurazioni come indirizzi API, nomi di modelli e API key dal contenuto dei componenti.

In un progetto React creato con Vite, le variabili che devono essere lette nel codice frontend devono iniziare con il prefisso `VITE_`. Si leggono tramite `import.meta.env`.

Il file `.env` viene creato nella cartella principale del progetto, accanto a `package.json` e non dentro la cartella `src`. Vite lo legge automaticamente quando il progetto viene avviato con `pnpm dev`, quindi non è necessario aggiungere un comando o una dipendenza specifica.

Se il file viene creato o modificato mentre il server è già attivo, occorre fermare il server e avviarlo di nuovo con `pnpm dev`.

esempio

File `.env`:

```
VITE_MISTRAL_API_KEY=la_tua_api_key
```

File `.gitignore`:

```
.env
```

Lettura della variabile nel codice JavaScript:

```
const apiKey = import.meta.env.VITE_MISTRAL_API_KEY;
```

Il file `.env` non deve essere caricato su GitHub se contiene chiavi private. Tuttavia, una variabile `VITE_` viene inclusa nel codice inviato al browser: per proteggere davvero una API key, la richiesta deve passare da un backend.

CAPITOLO 5

Organizzare le services

NOTA

Una service è un file che contiene funzioni dedicate alla comunicazione con un'API. Serve per separare la logica delle richieste dal componente React che mostra l'interfaccia.

Il componente gestisce input, bottoni e state; la service prepara la fetch e restituisce il dato necessario. Questa divisione evita di ripetere la stessa configurazione in più componenti.

Per organizzare le richieste si può creare una cartella `services` dentro `src`. Nel file `mistral.js` viene esportata la funzione `askMistral()`, che riceve soltanto la domanda e restituisce il testo generato.

esempio

Struttura del progetto:

```
src/  
  services/  
    mistral.js  
  App.jsx
```

File `src/services/mistral.js`:

```
export async function askMistral(question) {
  const response = await fetch(
    "https://api.mistral.ai/v1/chat/completions",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${import.meta.env.VITE_MISTRAL_API_KEY}`
      },
      body: JSON.stringify({
        model: "mistral-small-latest",
        messages: [
          {
            role: "user",
            content: question
          }
        ]
      })
    }
  );

  const data = await response.json();

  return data.choices[0].message.content;
}
```

File `src/App.jsx` :

```
import { useState } from "react";
import { askMistral } from "../services/mistral";

function App() {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");

  async function handleSubmit() {
    const result = await askMistral(question);

    setAnswer(result);
  }

  return (
    <div>
      <input
        type="text"
        value={question}
        onChange={(event) => setQuestion(event.target.value)}
        placeholder="Scrivi una domanda"
      />

      <button onClick={handleSubmit}>Invia</button>

      {answer && <p>{answer}</p>}
    </div>
  );
}

export default App;
```

L'istruzione **export** rende la funzione disponibile fuori dal file. L'istruzione **import** permette al componente di usare quella funzione. Il componente passa a **askMistral(question)** solo la domanda scritta dall'utente.

CAPITOLO 6

Mostrare un loader

NOTA

Un loader informa l'utente che una richiesta è ancora in corso. È utile perché una risposta AI può richiedere alcuni secondi: senza un feedback, l'interfaccia sembrerebbe non funzionare.

Per creare un loader si usa uno state booleano, cioè un valore che può essere `true` o `false`. Il valore diventa `true` prima della richiesta e torna `false` dopo il ricevimento della risposta.

Il componente può usare la service creata in precedenza. Prima di chiamare `askMistral()` imposta `loading` a `true`; dopo aver ricevuto il risultato, lo riporta a `false`.

esempio

```
import { useState } from "react";
import { askMistral } from "../services/mistral";

function App() {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");
  const [loading, setLoading] = useState(false);

  async function handleSubmit() {
    setLoading(true);
    setAnswer("");

    const result = await askMistral(question);

    setAnswer(result);
    setLoading(false);
  }

  return (
    <div>
      <input
        type="text"
        value={question}
        onChange={(event) => setQuestion(event.target.value)}
        placeholder="Scrivi una domanda"
      />

      <button onClick={handleSubmit}>Invia</button>

      {loading && <p>Mistral sta rispondendo...</p>}

      {answer && <p>{answer}</p>}
    </div>
  );
}

export default App;
```

Il loader è visualizzato con lo short-circuiting: `loading && <p>...</p>`. Quando `loading` vale `true`, React mostra il paragrafo; quando torna `false`, il paragrafo scompare.

CAPITOLO 7

Gestire errori con try catch

NOTA

`try` e `catch` permettono di gestire gli errori prodotti da un'operazione. Il codice che potrebbe fallire viene scritto dentro `try`; se si verifica un errore, JavaScript esegue il blocco `catch`.

Con una fetch, gli errori possono dipendere dalla connessione, dall'API key o dalla risposta del server. Il componente può salvare un messaggio di errore nello state e mostrarlo nell'interfaccia.

Per rilevare anche una risposta HTTP non valida, la service controlla `response.ok`. Se il valore è `false`, la funzione usa `throw new Error()` per generare un errore, che verrà intercettato dal `catch` nel componente.

esempio

File `src/services/mistral.js`:

```
export async function askMistral(question) {
  const response = await fetch(
    "https://api.mistral.ai/v1/chat/completions",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${import.meta.env.VITE_MISTRAL_API_KEY}`
      },
      body: JSON.stringify({
        model: "mistral-small-latest",
        messages: [
          {
            role: "user",
            content: question
          }
        ]
      })
    }
  );

  if (!response.ok) {
    throw new Error("La richiesta a Mistral non è riuscita");
  }

  const data = await response.json();

  return data.choices[0].message.content;
}
```

File `src/App.jsx` :

```
import { useState } from "react";
import { askMistral } from "../services/mistral";

function App() {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  async function handleSubmit() {
    setLoading(true);
    setAnswer("");
    setError("");

    try {
      const result = await askMistral(question);

      setAnswer(result);
    } catch {
      setError("Errore durante la richiesta. Riprova.");
    }

    setLoading(false);
  }

  return (
    <div>
      <input
        type="text"
        value={question}
        onChange={(event) => setQuestion(event.target.value)}
        placeholder="Scrivi una domanda"
      />

      <button onClick={handleSubmit}>Invia</button>

      {loading && <p>Caricamento in corso...</p>}

      {error && <p>{error}</p>}

      {answer && <p>{answer}</p>}
    </div>
  );
}
```

```
export default App;
```

Il blocco `catch` impedisce che un errore interrompa il normale funzionamento dell'interfaccia. Anche quando la richiesta fallisce, il loader deve tornare a `false`, altrimenti il messaggio di caricamento resterebbe visibile.